

UNIVERSIDADE FEDERAL DE SANTA MARIA
COLÉGIO POLITÉCNICO
TECNOLOGIA EM SISTEMAS PARA INTERNET

PLATAFORMA DE STREAMING DE MÚSICAS

Trabalho da disciplina de Banco de Dados II

Aluno: Marcelo Azevedo

Professor: Daniel Lichtnow

Santa Maria, RS

2026

1. INTRODUÇÃO

Este trabalho implementa um banco de dados NoSQL usando MongoDB para modelar uma plataforma de streaming de músicas, inspirada no Spotify. O domínio envolve três entidades principais — Artista, Música e Playlist — com dois tipos de relacionamento: 1:N entre Artista e Música, e N:N entre Música e Playlist.

O modelo foi implementado em duas versões distintas: Embedded (Versão 1) e Referenced (Versão 2). Cada abordagem possui características próprias que a tornam mais adequada para determinados cenários de uso, conforme detalhado nas seções seguintes.

2. MODELO CONCEITUAL

O diagrama abaixo representa o modelo conceitual do domínio, elaborado com a ferramenta brModelo, seguindo a notação Entidade-Relacionamento (ER):

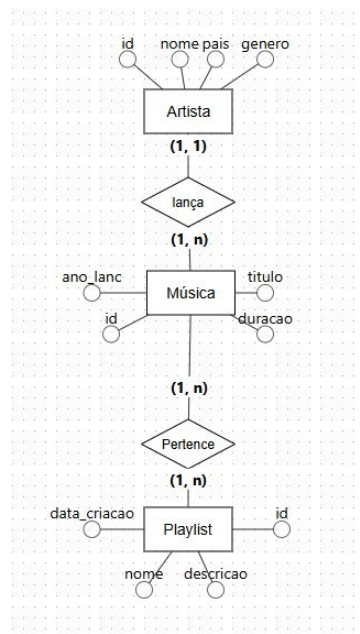


Figura 1 - Diagrama Entidade-Relacionamento

Entidade	Atributos
Artista	id, nome, pais, genero
Música	id, titulo, duracao, ano_lanc
Playlist	id, nome, descricao, data_criacao

Quadro 1 - Entidades e atributos do modelo

Os relacionamentos identificados são:

- Artista **lança** Música: relacionamento 1:N, onde um artista lança várias músicas;
- Música **pertence** a Playlist: relacionamento N:N, onde uma música pode estar em várias playlists e uma playlist contém várias músicas.

3. VERSÃO 1 — EMBEDDED RELATIONSHIPS

3.1 Estrutura

Na abordagem Embedded, os dados relacionados são armazenados diretamente dentro do documento principal, formando um único documento aninhado. As músicas ficam embutidas dentro do artista e as músicas de cada playlist ficam embutidas dentro da playlist:

```
artistas_v1 { _id, nome, pais, genero, musicas: [ { titulo, duracao,
ano_lanc } ] }
playlists_v1 { _id, nome, descricao, data_criacao, musicas:
[ { titulo, duracao, artista_nome } ] }
```

3.2 Justificativa

A abordagem Embedded é indicada quando os dados são sempre lidos juntos. No contexto deste trabalho, é muito comum buscar as músicas junto com o artista — por exemplo, ao exibir o perfil do artista com sua discografia completa. Com o Embedded, isso é feito em uma única leitura, sem necessidade de cruzar coleções.

Como o MongoDB armazena documentos em formato BSON, acessar dados aninhados é extremamente eficiente: tudo está no mesmo bloco de memória, sem operações de junção equivalentes ao JOIN relacional. Isso resulta em menor latência e maior throughput de leitura.

3.3 Vantagens e Desvantagens

- Leitura em um único acesso ao banco, sem necessidade de \$lookup;
- Melhor performance para leituras frequentes;
- Simplicidade na estrutura e nas consultas;
- Desvantagem: redundância de dados, pois a mesma música é copiada em cada playlist que a contém;
- Desvantagem: atualização custosa, pois alterar o título de uma música exige atualizar vários documentos.

4. VERSÃO 2 — REFERENCED RELATIONSHIPS

4.1 Estrutura

Na abordagem Referenced, cada entidade é armazenada em sua própria coleção, e os relacionamentos são feitos por meio de referências, equivalente às chaves estrangeiras do modelo relacional. A relação N:N entre Música e Playlist é resolvida com uma coleção intermediária:

```
artistas_v2      { _id, nome, pais, genero }
musicas_v2       { _id, titulo, duracao, ano_lanc, artista_id }
playlists_v2     { _id, nome, descricao, data_criacao }
musicas_playlists { musica_id, playlist_id }
```

4.2 Justificativa

A abordagem Referenced é indicada quando os dados são compartilhados entre múltiplas entidades e precisam ser atualizados de forma centralizada. No caso das músicas, uma mesma música pode pertencer a várias playlists. Com o Referenced, a música existe uma única vez na coleção `musicas_v2` e é referenciada quantas vezes for necessário, sem duplicação.

Essa abordagem também é mais adequada quando o conjunto de dados cresce de forma independente: uma playlist pode receber novas músicas sem que o documento da playlist precise ser completamente reescrito.

4.3 Vantagens e Desvantagens

- Sem redundância: cada música existe uma única vez na coleção;
- Atualização centralizada: alterar uma música reflete automaticamente em todas as playlists;
- Flexibilidade para relações N:N por meio da coleção intermediária;
- Desvantagem: requer \$lookup para cruzar coleções, equivalente ao JOIN relacional;
- Desvantagem: consultas mais complexas, com dois \$lookup em cascata para obter playlist e suas músicas.

5. COMPARATIVO ENTRE AS ABORDAGENS

Aspecto	Embedded (V1)	Referenced (V2)
Leitura	Um único documento	Requer \$lookup
Redundância	Alta	Nenhuma
Atualização	Custosa	Simples
Relação N:N	Dados repetidos	Coleção intermediária
Performance leitura	Alta	Média
Indicado para	Dados lidos juntos	Dados compartilhados

Quadro 2 - Comparativo entre Embedded e Referenced

6. CONSULTAS IMPLEMENTADAS

Foram implementadas duas consultas por versão, cada uma explorando um dos relacionamentos do modelo, utilizando o pipeline de agregação do MongoDB.

6.1 Consulta A — Artistas Rock com mais de uma música (1:N)

Retorna nome do artista, quantidade de músicas e duração média, apenas para artistas do gênero Rock com mais de uma música, ordenados pela duração média decrescente.

Na Versão 1 (Embedded): utiliza os operadores \$match, \$unwind, \$group, \$sort e \$project. O \$unwind expande o array de músicas embutido, transformando cada música em um documento separado para viabilizar o agrupamento.

Na Versão 2 (Referenced): utiliza \$lookup para juntar as coleções artistas_v2 e musicas_v2 antes do agrupamento, equivalente a um JOIN entre as tabelas.

6.2 Consulta B — Playlists de 2024 com música mais longa (N:N)

Retorna nome da playlist, quantidade de músicas e o título da música mais longa, filtrando apenas playlists criadas em 2024. A playlist 'Hard Rock Hits', criada em 2023, não aparece no resultado, demonstrando que o filtro por data está funcionando corretamente.

Na Versão 1 (Embedded): utiliza \$addField, \$sortArray e \$arrayElemAt para ordenar o array de músicas embutido por duração e selecionar a primeira (mais longa).

Na Versão 2 (Referenced): utiliza dois \$lookup em cascata — primeiro para atravessar a coleção intermediária musicas_playlists e depois para obter os dados de musicas_v2 — seguido de \$sort e \$group com \$first para selecionar a música mais longa.

7. CONCLUSÃO

As duas abordagens atendem ao mesmo domínio, mas com estratégias distintas. O Embedded prioriza a performance de leitura ao custo de redundância de dados, enquanto o Referenced prioriza a consistência e normalização ao custo de consultas mais complexas.

A escolha entre elas depende do padrão de acesso predominante da aplicação: se leituras são frequentes e os dados raramente mudam, o Embedded é a melhor escolha; se os dados são compartilhados entre múltiplas entidades e atualizados com frequência, o Referenced é mais adequado.

8. REPOSITÓRIO E EXECUÇÃO

O código completo do trabalho foi disponibilizado em repositório público no GitHub, contendo os scripts de inserção e consulta para as duas versões do modelo.

Repositório disponível em:

<https://github.com/mdazevedo96/DPADP0341---BANCO-DE-DADOS-II>

8.1 Estrutura do Repositório

```
|— README.md  
|— streaming_v1.js  
|— streaming_v2.js  
|— modelo_er.png
```

8.2 Como Executar

Pré-requisito: Docker Desktop instalado e em execução.

1. Baixar a imagem do MongoDB:

```
docker pull mongodb/mongodb-community-server:latest
```

2. Criar o container:

```
docker run --name mongodb -p 27017:27017 -d mongodb/mongodb-community-server:latest
```

3. No Docker Desktop, clicar nos três pontos ao lado do container mongodb → Open in terminal.

4. Dentro do terminal, conectar ao mongosh:

```
mongosh --port 27017
```


5. Executar a Versão 1 — digitar o comando abaixo e colar o conteúdo do arquivo `streaming_v1.js`:

```
use streaming_v1
```

6. Executar a Versão 2 — digitar o comando abaixo e colar o conteúdo do arquivo `streaming_v2.js`:

```
use streaming_v2
```

8.3 Scripts — Versão 1 (Embedded)

Criação das coleções e inserção dos artistas com músicas embutidas:

```
db.createCollection("artistas_v1")
db.createCollection("playlists_v1")
db.artistas_v1.insert({ _id: ObjectId("aaa000000000000000000001"),
  nome: "The Beatles",
  pais: "Reino Unido", genero: "Rock", musicas: [
    { titulo: "Let It Be", duracao: 243, ano_lanc: 1970 },
    { titulo: "Hey Jude", duracao: 431, ano_lanc: 1968 },
    { titulo: "Come Together", duracao: 259, ano_lanc: 1969 } ] })
db.artistas_v1.insert({ _id: ObjectId("aaa000000000000000000002"),
  nome: "Led Zeppelin",
  pais: "Reino Unido", genero: "Rock", musicas: [
    { titulo: "Stairway to Heaven", duracao: 482, ano_lanc: 1971 },
    { titulo: "Whole Lotta Love", duracao: 334, ano_lanc: 1969 },
    { titulo: "Black Dog", duracao: 296, ano_lanc: 1971 } ] })
db.artistas_v1.insert({ _id: ObjectId("aaa000000000000000000003"),
  nome: "Pink Floyd",
  pais: "Reino Unido", genero: "Rock", musicas: [
    { titulo: "Comfortably Numb", duracao: 382, ano_lanc: 1979 },
    { titulo: "Wish You Were Here", duracao: 314, ano_lanc:
1975 } ] })
db.artistas_v1.find().pretty()
```

Consulta A — artistas Rock com mais de uma música, ordenados por duração média decrescente:

```
db.artistas_v1.aggregate([
  { $match: { genero: "Rock" } },
  { $unwind: "$musicas" },
  { $group: { _id: "$nome", total_musicas: { $sum: 1 },
duracao_media: { $avg: "$musicas.duracao" } }},
  { $match: { total_musicas: { $gt: 1 } } },
  { $sort: { duracao_media: -1 } },
  { $project: { _id: 0, artista: "$_id", total_musicas: 1,
duracao_media: { $round: ["$duracao_media", 0] } } }
])
```

Consulta B — playlists de 2024 com música mais longa:

```
db.playlists_v1.aggregate([
  { $match: { data_criacao: { $gte: ISODate("2024-01-01"), $lt:
ISODate("2025-01-01") } } },
  { $addFields: { total_musicas: { $size: "$musicas" },
  musica_mais_longa: { $arrayElemAt: [
    { $sortBy: { input: "$musicas", sortBy: { duracao:
-1 } } }, 0 ] } } },
  { $project: { _id: 0, playlist: "$nome", total_musicas: 1,
  musica_mais_longa: "$musica_mais_longa.titulo" } }
])
```

8.4 Scripts — Versão 2 (Referenced)

Criação das coleções separadas com referências:

```
db.createCollection("artistas_v2")
db.createCollection("musicas_v2")
db.createCollection("playlists_v2")
db.createCollection("musicas_playlists")
db.artistas_v2.insert({ _id: ObjectId("aaa00000000000000000000000000001"),
nome: "The Beatles", país: "Reino Unido", genero: "Rock" })
db.artistas_v2.insert({ _id: ObjectId("aaa00000000000000000000000000002"),
nome: "Led Zeppelin", país: "Reino Unido", genero: "Rock" })
db.artistas_v2.insert({ _id: ObjectId("aaa00000000000000000000000000003"),
nome: "Pink Floyd", país: "Reino Unido", genero: "Rock" })
db.musicas_v2.insert({ _id: ObjectId("ccc00000000000000000000000000004"),
titulo: "Stairway to Heaven",
duracao: 482, ano_lanc: 1971, artista_id:
ObjectId("aaa00000000000000000000000000002") })
db.musicas_v2.insert({ _id: ObjectId("ccc00000000000000000000000000007"),
titulo: "Comfortably Numb",
duracao: 382, ano_lanc: 1979, artista_id:
ObjectId("aaa00000000000000000000000000003") })
// ... demais músicas no arquivo streaming_v2.js
db.musicas_playlists.insert({ musica_id:
ObjectId("ccc00000000000000000000000000004"), playlist_id:
ObjectId("bbb00000000000000000000000000001") })
// ... demais relações no arquivo streaming_v2.js
db.artistas_v2.find().pretty()
db.musicas_v2.find().pretty()
db.playlists_v2.find().pretty()
```

Consulta A — com \$lookup para juntar artistas e músicas:

```
db.artistas_v2.aggregate([
  { $match: { genero: "Rock" } },
  { $lookup: { from: "musicas_v2", localField: "_id", foreignField:
"artista_id", as: "musicas" } },
  { $unwind: "$musicas" },
  { $group: { _id: "$nome", total_musicas: { $sum: 1 },
duracao_media: { $avg: "$musicas.duracao" } } },
  { $match: { total_musicas: { $gt: 1 } } },
  { $sort: { duracao_media: -1 } },
  { $project: { _id: 0, artista: "$_id", total_musicas: 1,
duracao_media: { $round: ["$duracao_media", 0] } } }
])
```

Consulta B — com dois \$lookup em cascata pela coleção intermediária:

```
db.playlists_v2.aggregate([
  { $match: { data_criacao: { $gte: ISODate("2024-01-01"), $lt:
ISODate("2025-01-01") } } },
  { $lookup: { from: "musicas_playlists", localField: "_id",
foreignField: "playlist_id", as: "relacoes" } },
  { $unwind: "$relacoes" },
  { $lookup: { from: "musicas_v2", localField: "relacoes.musica_id",
foreignField: "_id", as: "musica" } },
  { $unwind: "$musica" },
  { $sort: { "musica.duracao": -1 } },
  { $group: { _id: "$nome", total_musicas: { $sum: 1 },
musica_mais_longa: { $first: "$musica.titulo" } } },
  { $project: { _id: 0, playlist: "$_id", total_musicas: 1,
musica_mais_longa: 1 } }
])
```

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S. B. *Sistemas de Banco de Dados*. 6. ed. São Paulo: Pearson Education, 2011.

GILLENSON, M. L. *Fundamentos de sistemas de gerência de banco de dados*. Rio de Janeiro: LTC, 2006.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistema de Banco de Dados*. 5. ed. Rio de Janeiro: Elsevier, 2006.

MONGODB, Inc. *MongoDB Documentation*. Disponível em: <https://www.mongodb.com/docs/>. Acesso em: 15 jun. 2026.

DOCKER INC. *Docker Desktop Documentation*. Disponível em: <https://www.docker.com/products/docker-desktop/>. Acesso em: 15 jun. 2026.

HEUSER, C. A. *Projeto de Banco de Dados*. 6. ed. Porto Alegre: Bookman, 2009.